

LAB#01

INTRODUCTION TO PYTHON, STRING, LOOP AND CONDITIONAL STATEMENTS

Familiarization with Python language and how to use different syntax to control indexing using strings, loop and conditional statement.

I. Introduction to Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is widely used for Artificial Intelligence, with packages for a number of applications including Machine Learning and NLP. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

a) Python Features:

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross- platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.

- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

b) Modules in Python:

- A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a file consisting of Python code. A module can define functions, classes and variables.
- In Python, modules are accessed by using the “import” statement. When you do this, you execute the code of the module, keeping the scopes of the definitions so that your current file(s) can make use of these.
- Many build-in modules of python, some of them are:
 1. Math
 2. Random
 3. Fraction
 4. Decimaletc. . .

c) Operator in Python:

- Basic algebraic operations
 - ✓ Four arithmetic operations: $a+b$, $a-b$, $a*b$, a/b
 - ✓ Exponentiation: $a**b$, where a is base value and b is exponent value.
 - ✓ Other elementary functions are not part of standard Python, but included in packages like NumPy and SciPy
- Comparison operators
 - ✓ Greater than, less than, etc.: $a < b$, $a > b$, $a <= b$, $a >= b$
 - ✓ Identity tests: $a == b$, $a != b$
- Bitwise operators
 - ✓ Bitwise or: $a | b$
 - ✓ Bitwise exclusive or: $a ^ b$ # Don't confuse this with exponentiation
 - ✓ Bitwise and: $a \& b$
 - ✓ Shift a left or right by b bits: $a << b$, $a >> b$

d) Input() Function:

The input() function pauses your program and waits for the user to enter some text. Once Python receives the user's input, it stores it in a variable to make it convenient for you to work with.

The purpose of an input statement is to get some information from the user of a program and store it into a Variable.

- Syntax: <variable> = input (<prompt>)

Example:

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 18:41:36) [MSC v.1900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> message = input("Tell me something, and I will repeat it back to you: ")
Tell me something, and I will repeat it back to you: Hello everyone!
>>> print(message)
Hello everyone!
>>> |
```

II. Strings

A *string* is simply a series of characters. Anything inside quotes is considered a string in Python, and you can use single, double or triple quotes around your strings like this:

"This is a string."

'This is also a string.'

"""This is also a string"""

a) Changing Case in a String with Methods:

One of the simplest tasks you can do with strings is change the case of the words in a string. Look at the following code, and try to determine what's happening:

```
>>> name = "string python"
>>> print(name.title())
String Python
>>>
```

b) Combining or Concatenating Strings:

When applied to strings, the + operation is called concatenation. It produces a new string that is a copy of the two original strings pasted together end-to-end. Notice that concatenation doesn't do anything clever like insert a space between the words. The Python interpreter has no way of knowing that you want a space; it does exactly what it is told.

```
>>> 'Hello'+'World'
'HelloWorld'
>>> 'Hi'+'Hi'+'Hi'
'HiHiHi'
>>> 'Hi'*3
'HiHiHi'
>>> |
```

Strings can be concatenated with the „+“ operator and repeated with „*“

Example:

```
>>> first_name = "sammy"
>>> last_name = "more"
>>> full_name = first_name + " " + last_name
>>> print("Hello, " + full_name.title() + "!")
Hello, Sammy More!
>>>
```

c) Indexing of string:

Python starts indexing at 0. A string *s* will have indexes running from 0 to *len(s)*-1 (where *len(s)* is the length of *s*) in integer quantities.

S	a	m	m	Y		S	h	A	r	k	!
0	1	2	3	4	5	6	7	8	9	10	11

Example:

- *s*[*i*] : fetches the *i*th element in *s*
- *s*[*i*:*j*] : fetches elements *i* (inclusive) through *j* (not inclusive)
- *s*[:*j*] fetches all elements up to, but not including *j*
- *s*[*i*:] fetches all elements from *i* onward (inclusive)
- *s*[*i*:*j*:*k*] extracts every *k*th element starting with index *i* (inclusive) and ending with index *j* (not inclusive)
- Python also supports negative indexes. For example, *s*[-1] means extract the first element of *s* from the end (same as *s*[*len(s)*-1])

```
ss="Sammy Shark! "
print(ss[2])
print(ss[6:11])
print(ss[:5])
print(ss[5:])
print(ss[0:5:2])
print(ss[-2])
```

Output:

```
m
Shark
Sammy
Shark!
Smy
k
>>> |
```

d) String Methods:

Some string methods are defined below:

- **strip()**: This method is used to **delete all the leading and trailing** characters mentioned in its argument.
- **lstrip()**: This method is used to **delete all the leading** characters mentioned in its argument.
- **rstrip()**: This method is used to **delete all the trailing** characters mentioned in its argument.
- **min("string")**: This function returns the **minimum value alphabet** from the string.
- **max("string")**: This function returns the **maximum value alphabet** from string.
- **continue**: The continue statement in Python is used to skip the remaining code inside a loop for the current iteration only.

III. Functions

Basically, we can divide functions into the following two types:

1. Built-in functions - Functions that are built into Python.
2. User-defined functions - Functions defined by the users themselves.

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. As you already know, Python gives you many built-in functions like print(), etc. but you can also create your own functions. These functions are called user-defined functions.

a) Defining a Function:

You can define functions to provide the required functionality. Here are simple rules to define a function in Python.

- Function blocks begin with the keyword **def** followed by the function name and parentheses (()).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The code block within every function starts with a colon (:) and is indented.

Syntax:

```
def function_name (PARAMETERS):  
    statement
```

Example:

```
def greet_user (username):  
    print ('Hello, ' + username.title() + '!')  
greet_user ('abc')
```

IV. Loop Statements

a) For loop Statement:

Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable. It has the ability to iterate over the items of any sequence, such as a list or a string.

Syntax:

```
for iterating variable in sequence:  
    statement(s)
```

Example:

```
fruits = ['banana', 'apple', 'mango']  
for fruit in fruits:  
    print ('Current fruit :', fruit)
```

Output:

```
Current fruit : banana  
Current fruit : apple  
Current fruit : mango  
>>> |
```

b) While Loop Statement:

The for loop takes a collection of items and executes a block of code once for each item in the collection. In contrast, while loop statement in Python programming language repeatedly executes a target statement as long as a given condition is true.

Syntax:

```
The syntax of a while loop in Python programming language is –  
while expression:  
    statement(s)
```

Example:

```
a = 0  
while a < 10:  
    a = a + 1  
    print ("Result",a)  
|
```

V. CONDITIONAL STATEMENTS

a) *If-Else Statement:*

An *if-else* block is similar to a simple if statement, but the else statement allows you to define an action or set of actions that are executed when the conditional test fails.

Example:

```
cars=['audi rr','bwm oo','subaru cc']
for car in cars:
    if car=='bwm':
        print(car.upper())
    else:
        print(car.title())
```

b) *The if-elif-else Statement:*

The elif statement allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE.

Syntax:

```
if expression1:
    statement(s)
elif expression2:
    statement(s)
elif expression3:
    statement(s)
else:
    statement(s)
```

Lab 01 : It is about data types, operators, input/output and conditional statements.

Activity 01:

You need to take initialize 4 different variables of different type and print them using f-string method and print method, Also find out the type of each variable.

Activity 02:

Draw the following pattern using single print statement.

```
*  
***  
*****  
*****  
*****
```

Activity 03:

Write a script that take user input for a number then adds 3 to that number. Then multiplies the result by 2, subtract 4, then again adds 3, then print the result.

Activity 04:

Write a script that takes input as radius then calculate area of circle. (Hint: $A = \pi r^2$)

Activity 05:

Write a Python script that asks users for their favourite color. Create the following output (assuming blue is the chosen color) (hint: use „+“ and „*“)

```
blueblueblueblueblueblueblueblueblueblue  
blue                                     blue  
blueblueblueblueblueblueblueblueblueblue
```

Not: it should work for all the inputs.

Activity 06:

Write a program that have a variable for the name of a city and its country. The program should print a simple sentence, such as "Karachi is in Pakistan". Also you need to set the default values for both the variables.

Activity 07:

Take a number from user as user_num. check if the number is odd, even. if it is odd then convert the number into even and if the number is even then convert it into odd. after and before the conversion print the number.

Activity 08:

Take user height and weight and calculate its BMI and also print the user health status.

Adult and Child BMI Calculator

Calculate BMI for Adult Age 20+ ▾

Weight: pounds ▾

Height: inches ▾

BMI	Status
≤ 18.4	Underweight
18.5 - 24.9	Normal
25.0 - 39.9	Overweight
≥ 40.0	Obese

Activity 09:

Take the number from user and always print its absolute value.

Activity 10:

Stages of Life: Write an if-elif-else chain that determines a person's stage of life. Set a value for the variable age, and then:

1. If the person is less than 2 years old, print a message that the person is a baby.
2. If the person is at least 4 years old but less than 13, print a message that the person is a kid.
3. If the person is at least 13 years old but less than 20, print a message that the person is a teenager.
4. If the person is at least 20 years old but less than 65, print a message that the person is an adult.
5. If the person is age 65 or older, print a message that the person is an old a message that the person is an old.